Flow diagrams — explanation and the rules behind them

Companion to `docs/architecture/diagrams/lossrun-flows.drawio` (five pages, open in draw.io / diagrams.net). Everything below was verified against the code on 2026-10-25 (`main` after brief 28); file references are given so the team can read the real thing. Numbers are the values in code or configuration today, not targets.

Legend used on every page: white = deterministic step (no model) · red = model call (metered) · amber diamond = decision · blue cylinder = store · orange = human step · green = published / straight-through · dashed purple = external system.

Page 1 — How a carrier format and the glossary come into existence

The one-sentence version for the team: the *glossary* answers "what does this header mean" and is global; the *format spec* answers "where and how does this layout print it" and is per carrier layout. A document is first matched against formats the system already knows; only when nothing matches does a model get asked, and nothing it proposes is used until it reproduces the document's own printed totals.

Steps in order

1. **Ingestion** (`src/LossRun.Ingestion/IngestionPipeline.cs`). Sniff, unlock, text-layer and scan-dpi check. Content Understanding does layout (single run) or classification + bundle split (multi-page). Output is `graph.json`: pages, words with polygons, paragraphs with roles, tables with cells. Spreadsheets are read natively — no model, no cost.
2. **Header resolution, before any model** (`src/LossRun.Contracts/ReferenceData.cs:77-220`, `CuResultMapper.cs:272-324`). Each header cell is resolved in this order: exact alias → component-sum ("Paid CLM+MED+EXP" → total.paid) → component + measure ("Med Pd") → conditional word ("Loss") → measure-only ("Paid" alone → total.paid at 0.8 confidence). Seed `glossary.json` (0.4.3, 55 fields, 11 ignored headers) is consulted first, then **active** learned aliases only. Header repair drops a form row above the table and splits merged cells when that increases resolution; stacked headers ("Claim no. over Claim type") resolve to two fields.
3. **Conditional aliases** are kept only when the table has the required sibling: "Loss Paid" means indemnity paid only if another column resolves to `claim.medical.*`; otherwise the header stays unresolved as *ambiguous-composite* and the document cannot publish unless printed totals tie.
4. **Scope gates** run before any learning: line of business ≠ WC → kept, not processed (ADR 0020); loss-analysis and changes-only pages → kept (ADR 0021); a narrative PDF with no tables → review, never learned (ADR 0009).
5. **Fingerprint** (`src/LossRun.Formats/Fingerprint.cs`). Two free keys: `identityKey` = SHA-256 of page-1 title/page-header text (digits stripped) and `headerKey` = SHA-256 of the first table's header cells. Exact match on both, on a non-retired version → deterministic hit (score 1.0, zero model calls). Identity match only → *drift* (the carrier re-arranged columns).
6. **Embedding tier** (PDF only, one embedding, `text-embedding-3-large` on 2,000 chars of page 1 with digits masked): cosine against each format's centroid must be ≥ min(adaptive threshold = max(0.90, mean – 2σ) once 5 hits exist, spec threshold default 0.95) **and** identity-token containment ≥ 0.8. Spreadsheets skip this tier.
7. **Recovery** (`FormatRouter.cs:355-412`): on a miss, replay every non-retired same-kind format whose carrier matches the submission hint or whose identity tokens are ≥ 50 % contained in page 1; keep the best by rank (3: no hard failures and no unmapped headers; 2: no hard failures; 1: only OCR-level failures). Still free.
8. **Replay** (`src/LossRun.Formats/Replay/ReplayEngine.cs`): tables qualify when ≥ 80 % of the spec's sources are found in the headers; scanned pages accept near-miss headers at similarity ≥ 0.8; every extracted value carries lineage (page + polygon or sheet + cell, `formatId@version`, confidence = lowest word confidence under the polygon). The full rule catalog runs. `ModelCalls` is always 0.
9. **Routing** (`FormatRouter.Decide`, chain on the diagram and on page 3): the outcome is ReplayAccepted, ReplayWithDelta, Learn or Review. Three post-checks demote an accepted replay to review: the format is known to print totals and none were read; an ambiguous composite column without a clean R-10/R-12 pass; a scan below 280 dpi going to learn.
10. **Delta** (`src/LossRun.Learning/FormatRepairer.cs`): drift with unmapped headers and ≤ 3 soft failures gets one small-model call (`gpt-5.4-mini`, schema `column_map_delta`) that may only add mappings for the unmapped headers; verified the same way; published as version n+1 inheriting the prior version's status. If it is not clean, one full learn with the prior spec as context; never for scans < 280 dpi.

11 **Learn** (`src/LossRun.Learning/FormatLearner.cs:52-152`):

- **Slot**: Durable entity `LearnGate` — `Learn__MaxConcurrent` slots (Terraform 1, code default 3), lease 15 min, denied callers wait on a 10-second durable timer. A document that waited re-routes first, so twenty documents of a new carrier produce one learn and nineteen replays.
- **Prompt** `learn-v11`: a byte-identical prefix (glossary fields, composite/conditional words, ignore list, rule catalog, strict JSON schema, a worked example) plus the document (first 3 pages' markdown ≤ 12k chars, header cells annotated with their glossary resolution, 8 sample rows) plus **instructions** derived from layout evidence (`LayoutEvidenceDetector`): records-rows / records-lines, measures-as-rows, stacked, claims-with-period-totals; the header-block fields; the prior spec on drift.
- **Model**: `gpt-5.4-mini@2026-03-17` (`config/models.json`; `gpt-5.2` when quota exists), Responses API with strict `json_schema`, prompt cache, 429 retries honouring `retry-after`. Metered: input, cached input, output tokens.
- **Lint autofixes** (no model; `SpecLint.cs`): ~25 deterministic repairs — locate printed constants, retarget row-level and coded columns, map lookup subtotals, relax total-row punctuation, keep printed zeros, drop unnamed components, declare absent components, declare `statusFrom` only when counts tie, locate header-block fields, and more — followed by lint **checks** that become issues (a header both mapped and ignored, two columns → one target, constants outside the allow-list…).
- **Verifier** (`SpecVerifier.cs`): replay the candidate on this document, run the catalog, compute required-field coverage (`learn-gate.json` 0.3.0) and the totals tie (R-10/R-12; R-26 for summary-only runs, where every printed amount must be covered by a check).
- **Gate**: refuse on any hard failure, any unmapped header, any lint issue, or a missing claim number; refuse an ambiguous composite without a tie; pass as `pdf-totals` / `spreadsheet-totals` when nothing is missing and the tie is true; `spreadsheet-rules` only for spreadsheets with coverage ≥ 0.8 and the carrier named by the submission.
- **Loop**: up to 5 attempts (`maxIterations.pdf/spreadsheet`), each failure fed back as a turn. Exhaustion keeps the best attempt by penalty as the reviewer's pre-fill, routes to document-review and raises a `learn-exhausted` (or `delta-exhausted`) escalation.

1 **Publish** (`FormatLearner.PublishAsync`): immutable version with fingerprint keys, an embedding centroid (one more embedding), the learning record (model, prompt version, iterations, tokens, gate). A new carrier's first version is **shadow**. Candidate glossary aliases are written for every column mapping the seed glossary did not already resolve.

2 **Activation**: a shadow format opens a confirm-format review on the learning document and on each later replay; "Confirm format and approve document" activates it (`approval:user`); "Approve document only" keeps it shadow; reject never activates. Three clean ReplayAccepted replays also activate it automatically; a review or hard failure resets the count. Retired versions are excluded from matching but kept for lineage.

Glossary alias lifecycle (ADR 0019)

State	Entered when	Effect
Candidate	a learn publishes a mapping the seed glossary does not already resolve to that field	never resolves headers; carries provenance (format, document, model, prompt version, totals tie)
Active	≥ 3 distinct documents across ≥ 2 distinct formats verified it (code default `AliasPolicy`), **or** a reviewer accepted / re-mapped it in the confirm-format review (untouched candidates of that format are implied accepts)	resolves headers for every carrier, after the seed
Retired	a reviewer rejected it, or a correction was made on a value read through it (audit-linked)	never re-learned

Hand base `glossary.json` changes only by pull request (tuning-loop tier 1); because it sits in the prompt prefix, a change requires the corpus v5 prompt gate.

Two gaps found while verifying: brief 21 asked for activation thresholds of 5 documents / 3 formats as configuration — the code still uses 3 / 2 as a hard-coded default; and `lossrun glossary export --promote` does not exist (export only). Both are small; listed under "follow-ups" at the end.

Page 2 — Durable Functions, Durable Task Scheduler, multi-file synchronisation

What runs where

- **HTTP API** (`src/LossRun.Functions/HttpApi.cs:24-144`, `UploadApi.cs:58-128`): authenticates, raises the body cap to 100 MB for this route (413 above), buffers the files, derives `uploadId = up-` + SHA-256(`key:{user}:{Idempotency-Key}` or `content:{user}:{file:sha…}`)[..20], writes the sources to Blob `uploads/{uploadId}/{file}` (no overwrite), creates `dbo.Upload` with status `received` and an `upload.create` audit row, and schedules `IngestUploadOrchestrator` with **instance id = uploadId**. A second POST of the same package finds the instance and returns `existing: true`. Response 202 with status and SSE URLs.
- **IngestUploadOrchestrator** (`Orchestrations.cs:85-151`): `ExpandUpload` → `Task.WhenAll(ParseFile × N)` → `RecordItems` → `Task.WhenAll(ProcessLossRun × M)` with sub-instance ids `{uploadId}~{documentId}` → `CompleteUpload` → review wait loop. Any `TaskFailedException` → `FailUpload` (status failed, `unhandled-exception` escalation) and the orchestration ends Failed.
- **ProcessLossRun** (`Orchestrations.cs:157-209`): `Route` → for Learn/Delta tiers acquire a `LearnGate` slot (re-route after waiting) → `Learn`/`Delta` → `ValidateAndPersist` → returns `DocumentOutcome`.
- **Durable Task Scheduler** (`infra/orchestration.tf`, `host.json`): `Microsoft.DurableTask/schedulers`, Consumption SKU, task hub `lossrun` (dev hub `lossrun-dev`), storage provider `azureManaged`. It stores every scheduled/completed activity and sub-orchestration event, custom status JSON (what the SSE stream shows), durable timers (the 7-day review deadline, the 10-second LearnGate retry), pending external events (`ReviewDecision`) and the `LearnGate` entity state. A crashed worker is redelivered the work item; completed activities are not re-run; in-flight ones are — which is why every activity is idempotent (`PersistExtraction` returns early if the document exists; `RecordItems` upserts; `OpenReview` inserts only if absent; blobs are rewritten identically; a held learn slot is reclaimed after the 15-minute lease).
- **Functions host**: Flex Consumption FC1, .NET 10 isolated, 2 GB instances, max 40; `always_ready` ×1 for http and durable only in demo mode (avoids the ~15 s cold start); `functionTimeout` 30 min. No Durable retry policies on activities; retries live lower (model client 8×, EF execution strategy on SQL transactions).

Synchronisation and consistency

Concern	Mechanism	Where		
Upload status	a stored manifest field (`received → processing → awaiting-review \	completed \	failed), updated by activities; read-modify-write under UPDLOCK, HOLDLOCK on dbo.Upload, so parallel ValidateAndPersist` calls of one upload serialise	`SqlRecordStore.cs:33-61`
Per-document progress	custom status of each sub-orchestration, polled every 500 ms by the SSE endpoint for ≤ 200 s (front end reconnects at 230 s)	`HttpApi.cs:156-202`, `UploadApi.cs:148-163`		
Model concurrency	`LearnGate` entity: slots, 15-minute lease, 10-second durable-timer retry, release in `finally`	`LearnGate.cs`, `Orchestrations.cs:212-240`		
Review decisions	external event `ReviewDecision` raced against a 7-day timer; an event for a document not in `pending` is ignored; `POST /reviews/{id}/decision` answers 409 if the item is no longer pending	`Orchestrations.cs:118-146`, `UploadApi.cs:280-283`		

Concern	Mechanism	Where		
Concurrent writes to one document	`dbo.ApplyDecision`: one round trip, `UPDLOCK` on the document row, refuses a stale revision (returns 0), writes Override, AuditEvent, Claim history, RuleOutcome, Document, ReviewItem and Outbox rows, bumps the quote revision, invalidates feature snapshots — all in one transaction	`db/changelog/0016-apply-decision-proc.sql`		
Direct override vs orchestration	409 `in-review` while a document-review is pending; otherwise up to 5 attempts with jittered back-off on a revision conflict	`DocumentCommands.cs:78-138`		
Idempotency keys	upload: header or content hash → deterministic uploadId; document: `doc-` + SHA-256(`uploadId:fileSha`); outbox id = Service Bus `MessageId` = CloudEvent `id` = `Idempotency-Key` header to the quote system	`UploadApi.cs:93-96`, `IngestionPipeline.cs:197-199`, `Publishing.cs`, `Dispatcher.cs:26-27`		
Outbox	rows written in the same SQL transaction as the audit row (or inside `dbo.ApplyDecision`); fast path publishes immediately (unawaited); a 30-second sweep republishes rows older than 20 s; publish first, mark `PublishedUtc` second	`ClaimCommandService.cs:275-300`, `OutboxRelay.cs`, `IntegrationFunctions.cs:19-34`		
Ordering at the receiver	Service Bus sessions per quote (FIFO), duplicate detection 1 day, max delivery 10, DLQ; the receiver refuses a lower `quoteRevision` with 409 `stale-revision`, which the dispatcher records as `superseded` and does not retry	`infra/integration.tf`, `Dispatcher.cs:99-113`, ADR 0024		
First push to the quote system	written only when no document of the upload is `awaiting-review`; a `document-review` holds it, a `confirm-format` does not	`Activities.cs:399-428`, `:465`		

Page 3 — Validation engine

Mechanics

- The catalog (`docs/evaluation/rule-catalog.json`, 0.11.0) is **data**, embedded into `LossRun.Rules` and executed by a generic evaluator (`RuleEvaluator.cs`) — there is no per-rule code. One outcome per rule × scope instance (document, each period, each claim) with status Pass / Fail / NotApplicable / NotInformative, severity, message and the canonical paths it read (`affectedFields`).
- **Three-valued logic**: a comparison with a null operand is unknown → NotApplicable. Missing data is scored by completeness, never as a failure. A pass becomes *not-informative* when a value it read was derived from the other values it read (incurred = paid + reserve where incurred itself was derived as paid + reserve).
- **Severity**: hard blocks straight-through and is counted in the first-pass rate; soft is a warning (and counts toward the "≤ 3 soft" delta threshold); flag is a routing/analytics signal, never counted.
- Amount comparisons carry a $1 tolerance; dates compare exactly.

Stages

(a) ingestion checks — no rules; (b) learn gate — before a spec may publish; (c) validation on every document, replay hits included — stored in `ResultJson` and `dbo.RuleOutcome` per revision, shown in the Rules dock with click-to-source; (d) after an override — value set, lineage `manual`, derived sums recomputed, catalog re-run, revision + 1; (e) field status and readiness — derived, never stored: reject → rejected; pending → unresolved; R-33 required and null → unresolved; manual override → approved; hard failure touching the path → unresolved(ruleId); else approved; readiness = no pending documents and no unresolved fields; (f) escalation sweep — immediate triggers and hourly patterns (ADR 0025).

The rules

The table on the diagram page groups R-01…R-33 by what they check. Highlights the team should know:

- **Ties to printed totals** (R-10, R-12, R-26) are the system's oracle: they are the learn gate, the alias-verification evidence and the strongest routing signal. Kept rows (other lines of business) are allowed in the tie (ADR 0020 amendment).
- **R-14** is per policy period since ADR 0016; **R-27/R-28** flag the two ways a "no losses" statement can be mis-read.
- **R-20/R-29** become hard when the document declares expense as separate.
- **R-33** is the only quote-level rule; it never routes a document, only blocks readiness and nulls the value in the verified-only view.
- **R-15 and R-25** need a `RuleContext` (target effective date, requested years) that no production path supplies today — they always evaluate NotApplicable. Worth a follow-up.

Routing chain

Thirteen ordered conditions plus three post-checks (listed on the page). The final tier determines the manifest status: ReplayAccepted / Learned / Delta → processed; Review → awaiting-review and the quote push waits. "Kept, not processed" is decided only at stage (a), never by a rule.

Pages 4 and 5 — End to end

Both pages read top-to-bottom, column = component. Shared prefix: upload → 202 → orchestration → ingestion (Content Understanding pages metered per `UsageEvent`) → fingerprint.

Case A (new carrier format): fingerprint miss (1 embedding) → recovery fails → learn (1–5 calls) → publish v1 as shadow (+1 centroid embedding) → validate and persist (lineage row per value, audit `extraction.persist`, `format.learned` event, confirm-format review item) → `CompleteUpload` bumps the quote revision to 1 and pushes `quote.payload` with the document listed under `excluded[]` as pending-review (no claims yet) → the reviewer confirms the format → `dbo.ApplyDecision` (revision 2, `format.approve` audit, aliases activated) → `quote.payload` at revision 2 with the claims → Service Bus → `PUT quotes/{q}` → the quote system fetches `?revision=2` → the Loss Information page updates (measured: 1 s after the decision).

Case B (active format): fingerprint hit (free) → replay (0 model calls, p50 0.3 s) → ReplayAccepted → persist → `CompleteUpload` pushes revision 1 **with** the claims and both feature snapshots → same downstream chain. No human unless a routing rule sends the document to review.

Measured difference (results `2026-10-24-learn-loop-quality`): learned path mean $0.051 and 40 s p50 per document; replay $0.0087 and 0.3 s; spreadsheets $0.

Every store and who writes it

Blob: `uploads/` (sources), `extractions/…/graph.json`, `routing.json`, `replay.json`, `formats/{id}/{v}.json` + `index/stats/centroid.json` (cache under the SQL registry), `quote-system-mock/…`. SQL (`dbo`): Upload, UploadItem, UsageEvent (ledger), Document, PolicyPeriod, Claim (+history), ExtractedValue (ledger), RuleOutcome, DocumentCost, AuditEvent (ledger), ReviewItem, Format, FormatVersion, FormatFieldStats, GlossaryAlias (+history), GlossaryAliasEvidence (ledger), Outbox, QuoteRevision (+history), FeatureSnapshot, Override (ledger), QuoteContext, Escalation. Others: Durable Task Scheduler (instances, entity, timers), Storage queue `Chat:IndexQueue`, AI Search index `lossrun-docs`, Service Bus `quote-commands`, Event Grid topic.

Decision records these diagrams rest on

Already written: 0007 verification before publish · 0009 segment without a table → review · 0014 outbox / Service Bus / Event Grid · 0016 R-14 per period · 0017 summary-only runs · 0018 layout constructs and learn repairs · 0019 learned glossary aliases · 0020 line of business · 0021 analysis and changes-only pages · 0022 report-only claims · 0023 header-block fields · 0024 quote revision and receiver contract · 0025 engineering escalation.

Proposed ADR 0026 — where carrier knowledge lives (not yet written; the sentence the team should carry): *meaning is global, layout is per format.* A header-to-field mapping belongs to the glossary when it is true of the words regardless of who printed them, and to the format spec when it is true of one layout; a mapping learned for one format becomes global only with evidence across formats or a reviewer's confirmation; a mapping whose meaning depends on the table is a conditional alias with the condition attached; no code may branch on a carrier's identity (the generic guard enforces this).

Follow-ups found while drawing

1. Alias activation thresholds (5 documents / 3 formats, as agreed in brief 21) are still the code default 3 / 2 — make them configuration.
2. `lossrun glossary export --promote` was never implemented; promotion is by pull request only — either implement or correct the docs.
3. R-15 / R-25 are dead in production without a `RuleContext`; wire the quote's target effective date and requested years from `QuoteContext`.
4. `config/models.json` runs `gpt-5.4-mini` for learn; CLAUDE.md still says the default is `gpt-5.2` — align the text with ADR 0004.